The Journal of Systems and Software 237 (2026) 112830

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

RunSoC: A design-space exploration framework for scheduling and allocation of automotive software tasks to semiconductor design partitions in system-on-chips

Lucas Mauser [a,b,*], Xinyu Zhang [b], Daniel Krüger [b], Moritz Wäschle [a], Stefan Wagner [b]

[a] Daimler Truck AG, Stuttgart, Germany
[b] Technical University of Munich, Heilbronn, Germany

ARTICLE INFO

Editor: Guest Editor

Keywords:
Automotive systems
Mixed-criticality systems
HPC platform
Multi-core system-on-chip
Task scheduling
FCFS and PAS
Task allocation
Static and dynamic allocation

ABSTRACT

Background: Automotive systems are transitioning from microcontroller-based architectures to high-performance multi-core System-on-Chip (SoC) platforms to meet the demands of software-defined vehicles. This shift complicates the scheduling and allocation of software tasks, making it challenging to balance performance, timing demands, determinism, adaptability, and efficiency using current static or dynamic allocation methods.
Aim: This paper presents RunSoC, a design-space exploration framework for scheduling and allocation strategies of automotive software tasks on multi-core SoCs. Rather than targeting deployable production schedules, the framework enables system architects to explore the impact of architectural choices under constrained computational resources.
Method: We conduct a rapid literature review and expert interviews to identify research gaps and industrial needs. Based on these insights, we design and validate a hybrid framework that integrates static and dynamic core allocation with two scheduling policies: first-come first-serve and priority-aware.
Results: We validate the framework on directed acyclic graph models of Advanced-Driver Assistance Systems (ADAS) workloads. The results highlight the high per-core efficiency and comparable average latency of static allocation, though it carries the risk of incomplete execution of event-driven task chains. Here, incorporating additional cores or dynamic allocation can enhance the overall utilization and throughput.
Conclusion: RunSoC provides a flexible, metrics-driven framework that integrates task dependencies and priorities, offering reproducible, performance-oriented scheduling and allocation analysis for early architectural decision-making in software-defined vehicle platforms.

1. Introduction

In the automotive industry, the Software-Defined Vehicle (SDV) drives a shift from microcontroller-based (μC-based) to microprocessor-based (μP-based) computing on homogeneous and heterogeneous multi-core System-on-Chip (SoC) architectures (see Fig. 1) (Mauser et al., 2025; Hammond et al., 2015; Gupta et al., 2019; Bateni and Liu, 2019; McLean et al., 2022; Dendaluce, 2015; Rettkowski et al., 2015). This shift is the result of many factors. The image processing algorithms used by Advanced Driver Assistance Systems (ADAS) on their way to autonomous driving require high computational capabilities (Waez et al., 2014; Saussard et al., 2015). Also, since features are increasingly interdependent, system consolidation is intended to reduce complexity, as well as development, material costs, and more (Vogelsang, 2020; Dendaluce, 2015; Dong et al., 2020). In addition to that, the SDV paradigm to split think (compute) and sense/act (I/O) further drives this consolidation in the form of the centralization of modern automotive Electric/Electronic (E/E) architectures (Mauser and Wagner, 2024; Gala et al., 2022). Here, vehicle domains such as ADAS, In-Vehicle Infotainment (IVI), Instrument Cluster (IC), powertrain, cabin, or chassis are consolidated within a few, powerful High-Performance Computers (HPCs) (Mauser et al., 2025; Gala et al., 2022; Dong et al., 2020). Given that each domain has distinct requirements in terms of safety, performance, updateability, and other factors, the consolidation of these domains results in a mixed-criticality HPC platform realized by homogeneous and heterogeneous multi-core SoC architectures (Mauser et al., 2025; Gupta et al., 2019; Dendaluce, 2015; Arndt et al., 2019). Although software is intended to become the key differentiator for Original Equipment Manufacturers (OEMs) (Mauser et al., 2025; Staron, 2021), it demonstrates that the foundation of the SDV is hardware-enabled (Tatar et al., 2024; Li et al., 2019). This creates the need for hardware-software co-design (Kempf et al., 2023; Tatar and Bayar, 2023) and poses the

* Corresponding author.
E-mail address: lucas.mauser@daimlertruck.com (L. Mauser).

https://doi.org/10.1016/j.jss.2026.112830
Received 12 October 2025; Received in revised form 15 January 2026; Accepted 22 February 2026
Available online 3 March 2026
0164-1212/© 2026 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

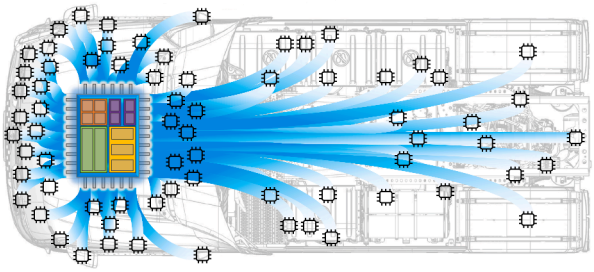


Fig. 1. From µC-based to SoC-based computing.

challenge of automotive software component task scheduling and allocation across multiple cores.

This challenge necessitates a systematic scheduling and allocation framework that considers the diverse functional and non-functional requirements of mixed-criticality HPC platforms. Motivated by this context, we pose our primary research question:

RQ: *How can automotive software tasks of an HPC be systematically mapped to the semiconductor design partitions of the HPC's SoC to meet the respective requirements?*

To address the research question, we conduct a Rapid Review (RR) of relevant literature combined with expert talks within the Daimler Truck AG. Based on the identified academic gaps and industry needs, we develop a design-space exploration framework to assist automotive software architects to analyze task scheduling and allocation strategies under constrainted computational resources. We validate the framework based on ADAS use cases. The combination of the RR and framework development, together with repeated expert talks, bridges academic research and industry practice.

The paper is organized as follows: We introduce the fundamental concepts and terminology necessary to understand this work in Section 2. In Section 3, we present the theoretical approach. Based on the results, Section 4 discusses related work, challenges and limitations of existing methods. Section 5 presents the proposed framework. Section 6 assesses the framework's performance based on ADAS use cases. Addressing the threats to validity in Section 8, we summarize the main findings and provide an outlook for future research in Section 9.

2. Background

This section introduces the fundamental concepts and terminology necessary to understand this work.

2.1. Software components according to AUTOSAR

AUTomotive Open System ARchitecture (AUTOSAR) defines a standardized layered software architecture for automotive Electronic Control Units (ECUs), ensuring portability, modularity, and well-specified interfaces between applications and hardware (AUTOSAR, 2025). Its three main software entities – runnables, tasks, and applications – form a hierarchy governing software organization, execution, and scheduling (Khenfri et al., 2020).

Runnables are the smallest execution units within a software component, which implement specific functionality. They are triggered and coordinated by the Run-Time Environment (RTE), which manages communication and execution flow, rather than being scheduled directly by the OS (Warschofsky, 2009). Tasks are the smallest schedulable entities in an AUTOSAR OS, which provide the execution context for one or more runnables. Tasks may be periodic or event-driven, and the OS scheduler assigns priorities and ensures timely completion according to real-time constraints (Khenfri et al., 2020). Applications group multiple tasks at the highest level. Tasks within an application can have distinct priorities. Their associated runnables may be distributed to enable simultaneous execution when hardware permits (Navet and Simonot-Lion, 2017).

We henceforth refer to tasks, since scheduling and allocation occur at the task level. This implicitly includes the runnables they execute.

2.2. Directed acyclic graph (DAG)

A Directed Acyclic Graph (DAG) is a fundamental abstraction widely applied in scheduling, data-flow analysis, and task-dependency modeling. It offers a natural framework to represent computations composed of interdependent tasks (Skiena, 2008). We employ DAGs to model the workload in the validation use cases and to provide the frontend representation of the framework.

A directed graph $G = (V, E)$ consists of a finite set of vertices $V = \{v_1, v_2, \ldots, v_n\}$ and a set of directed edges $E \subseteq V \times V$, where each edge $(v_i, v_j) \in E$ indicates a directed connection from v_i to v_j (Thulasiraman and Swamy, 1992). The graph is acyclic if no sequence of edges leads from a vertex back to itself while following edge directions. In other words, it contains no directed cycles (Thulasiraman and Swamy, 1992). Otherwise, it is cyclic. A path is a sequence of edges where every edge's end vertex is the start vertex of the next (Thulasiraman and Swamy, 1992). The critical path is the longest path and is used to analyze and control the total time required for project completion (Kelley and Walker, 1959).

2.3. Static and dynamic allocation

Static and dynamic allocation represent the predominant categories of task allocation methods and form the conceptual foundation of our framework (Rettkowski et al., 2015).

Static Allocation allocates tasks to cores during the design phase. As an offline strategy, it determines core usage before execution, based on prior knowledge of the DAG and task execution times. Since allocation is fixed once, it ensures high predictability with minimal runtime overhead (Gupta et al., 2019). Dynamic Allocation allocates tasks to cores at runtime.Unlike static allocation, it adapts core usage in response to the evolving execution state, such as task completions and changes in task eligibility. This flexibility allows allocations to be reevaluated at discrete decision points during execution (Rettkowski et al., 2015).

2.4. Memory and performance metrics

Efficient use of computational resources is critical to system performance. Key metrics include overhead, throughput, and latency, which are essential for our later discussion of performance trade-offs and optimizations.

Overhead refers to the additional utilization of computational or memory resources beyond core operations, such as context-switching or protocol handling. Excessive overhead can significantly reduce efficiency (Tanenbaum and Bos, 2015). Throughput measures the rate at which a system processes data or completes tasks, typically in operations or bits per second. High throughput indicates strong performance but must be balanced with resource limits (Stallings, 2016). Latency is the delay between a request and its response, often reflecting data access time in memory systems. In real-time applications, such as automotive safety, low latency is critical to system reliability (Laplante, 2004).

2.5. Amdahl's law

In computer science, Amdahl's law expresses the theoretical speedup of a task when additional resources, such as cores, are allocated (Hill and Marty, 2008). A task can be divided into a serial fraction f_{ser}, which does not benefit from additional resources, and a parallel fraction f_{par}, which scales with the number of available cores.

Let T denote the total execution time prior to resource enhancement. According to Amdahl's Law (Hill and Marty, 2008), the achievable

speedup S on N identical cores is:

$$S = \frac{1}{f_{ser} + \frac{f_{par}}{N}}. \quad (1)$$

As $N \to \infty$, the maximum attainable speedup is limited by the serial fraction (Amdahl, 1967) and given by:

$$S_{\max} = \frac{1}{f_{ser}}. \quad (2)$$

An understanding of Amdahl's law enables us to assess the theoretical upper bound on parallel speedup and to validate our framework's task allocation across multiple cores.

Building upon this background knowledge, the subsequent sections present the research methodology and related work that together provide the scientific foundation for our framework.

3. Research method

As a research method, we combine academic insights based on an RR according to the guidelines of Cartaxo et. al (Cartaxo et al., 2020) with industry perspectives based on repeated expert talks within the Daimler Truck AG. Doing so, we aim to "reduce the gap between academic research and software engineering practice" (Cartaxo et al., 2020).

3.1. Rapid review

An RR is a lightweight, practice-oriented form of Systematic Literature Reviews (SLRs) developed to address time and resource constraints. While SLRs are comprehensive and require significant collaboration and investment, RRs can be conducted by single researchers in close collaboration with practitioners, making them faster and more aligned with industry needs. This approach enables the synthesis of more up-to-date data and requires fewer resources. For this study, a predefined review protocol – derived from the guidelines of Cartaxo et al. (2020) to fit the research context and reduce bias – is used to ensure systematic and reproducible results. The following subsections highlight the review protocol's key milestones while methodological limitations are discussed in Section 8. The working products of the key milestones are documented and publicly available on GitHub and Zenodo. It allows researchers and practitioners to access more details and build their research upon our work.

3.1.1. Research question

The research question lays the foundation for systematic research (Brereton et al., 2007) and is an essential part of systematic reviews (Kitchenham et al., 2007). According to Cartaxo et al. (2020), the research question should be exploratory and narrower in contrast to SLRs. With a focus on RRs, research questions should help practitioners to solve their problems. As discussed in Section 1, this study aims to identify systematic approaches to schedule and allocate automotive software tasks on multi-core SoCs. We align this objective with the experts of Section 3.2.2 and formulate our research question.

3.1.2. Search strategy

In line with the constraints of an RR, we use the performance indicators *Recall*, *Precision*, and *F-measure* to set up an appropriate and efficient search strategy. Recall ensures the comprehensiveness of the review by minimizing the risk of overlooking relevant studies. Precision ensures that the identified studies will contribute to answer the research question. Here, the F-measure balances both metrics as the harmonic mean of precision and recall (Mourão et al., 2020).

Mourão et al. (2020) performed three SLRs in the field of software engineering. They compared the performance of nine digital libraries for the three SLRs using the performance indicators introduced above. Among the nine libraries, Scopus achieved the highest F-measure in each of the three SLRs. It achieved the highest precision and recall in two of the three SLRs, and the second highest precision and recall in the third SLR. Mourão et al. also investigated four hybrid strategies based on a Scopus database search combined with forward and backward snowballing in different orders. While this increases recall, the results indicate a reduction in precision, meaning the F-measure of the database-only search is consistently higher among the three SLRs. Given the cost and time constraints of RRs, we decide to perform the search without snowballing to target efficiency and precision over recall.

3.1.3. Search string

For the Scopus database search, we construct a search string based on the PICOC criteria (Population, Intervention, Comparison, Outcomes, Context) by Kitchenham et al (Kitchenham et al., 2007). The PICOC criteria decompose the research question to help refine the scope of the research, consider different viewpoints, and reduce bias.

P: Automotive software tasks – **I:** Systematic allocation of automotive software tasks to SoC design partitions – **C:** Systematic allocation approaches **OR** How this is currently performed in the automotive embedded systems domain – **O:** Framework for allocating tasks to semiconductor designs – **C:** Software-defined transformation towards mixed-criticality HPCs in automotive industry.

Based on the PICOC criteria, we created a preliminary search string. The authors discussed, reviewed, and modified the initial prototype to ensure its plausibility and reduce bias. Pilot searches confirmed its accuracy. We finally developed the following search string:

TITLE-ABS-KEY (software AND (vehicle OR automotive) AND ("System-on-Chip" OR "SoC" OR "semiconductor design partition*" OR microprocessor OR microcontroller) AND ("mapping" OR "allocation" OR "systematic distribution" OR method*) AND ("mixed critical*" OR "functional domain*" OR "real-time*" OR "safety-critical" OR "performance optimization" OR hpc OR "High*Performance Comput*" OR heterogene*s) AND NOT batter*)

3.1.4. Inclusion and exclusion process

To generate a seed set of studies from the Scopus database search, we define the inclusion and exclusion criteria presented in Table 1. For the inclusion criteria, we focus on content-related criteria. For the exclusion criteria, we focus on quality-related criteria. This differentiation ensures to achieve a certain level of quality, as discussed in the guidelines of Kitchenham et al. (2007). A study must match the content-related and quality-related criteria to be included. We derive the inclusion criteria from the research question and the PICOC criteria to consider only studies relevant to the objectives of this study. First, we exclude duplicates and papers without full-text availability. To ensure credibility, we include only peer-reviewed publications. Books and book chapters are excluded. The search, conducted in February 2025, is restricted to works published between 2015 and February 2025 to reduce irrelevant results and focus on recent advances. This timeframe is chosen to match the inclusion criteria and to reflect the increase in research on scheduling and allocation methodologies for software tasks to SoCs, as identified in the Scopus database. Papers in languages other than English, German, or Chinese are omitted, given the proficiency of the reviewers.

We use the Rayyan web application (Ouzzani et al., 2016) to streamline the literature screening process by employing semi-automated procedures. After the use of Rayyan's detection tool to remove duplicates, we exclude books and book chapters, then apply a year filter to retain publications from 2015 onward. Two independent reviewers manually screen the remaining records using Rayyan's blind mode to classify each item as *Include*, *Maybe*, or *Exclude* and to provide a justification for every decision. Then, blind mode is disabled, and any disagreements or *Maybe* cases are resolved through discussion to reach a consensus.

The initial literature search yields 192 results, reduced to 184 after removing duplicates, and to 178 after excluding unsuitable literature types. Applying the publication date filter removes 68 papers, leaving

Table 1
Inclusion and exclusion criteria.

A study is included if	A study is excluded if
IC1: the paper describes properties of SoC design partitions	**EC1**: the paper does not meet any of the inclusion criteria
IC2: the paper describes properties of automotive software tasks	**EC2**: the paper is duplicated
IC3: the paper describes a practice to map automotive software tasks on SoC design partitions	**EC3**: the paper is not peer-reviewed
	EC4: the paper is either a book or a chapter from a book
	EC5: the paper is published before 2015
	EC6: the paper is not written in English or German or Chinese
	EC7: the paper does not have available full text

Table 2
Data Extraction Criteria (DEC).

ID	DEC
1	Does the study identify the HPC approach (μC → SoC) as an enabler / accelerator to reduce today's system limitations?
2	Which SoC design and properties does the study investigate / elaborate?
3	Which software task properties / requirements are analyzed in the study?
4	Which technologies, methodologies or frameworks are used to map software tasks to semiconductor design partitions?
5	Which are the key challenges in allocating software tasks to semiconductor design partitions?
6	Which validation methods or case studies are presented to demonstrate the effectiveness of the proposed approach?
7	Are any industry standards or best practices followed in the allocation process?

110 for manual screening. Of these, 88 are excluded for irrelevance, resulting in 22 studies matching the research scope. The Sankey diagram and its related performance indicators recall, precision, and F-measure are publicly available on GitHub and Zenodo.

The following set contains the included studies to provide a central overview: (Hammond et al., 2015; Suluhan et al., 2024; Kempf et al., 2023; Tatar and Bayar, 2023; Gupta et al., 2019; Brown et al., 2019; Tatar et al., 2024; Bateni and Liu, 2019; McLean et al., 2022; Wang et al., 2023; Gala et al., 2022; Li et al., 2019; Waez et al., 2014; Schnarz et al., 2015; Saussard et al., 2015; Dendaluce, 2015; Dong et al., 2020; Hemmati et al., 2019; Goens et al., 2016; Arndt et al., 2019; Mack et al., 2023; Rettkowski et al., 2015).

3.1.5. Data extraction process

We break down the research question into the Data Extraction Criteria (DEC) presented in Table 2 to extract the data of the included papers. These criteria facilitate and unify the data extraction process. A comprehensive examination of the full text of each study is conducted to answer each of the DECs.

Section 4 presents the extracted data as related work. Combined with the findings from the expert interviews in the subsequent section, this forms the foundation for the elaboration of the proposed framework.

3.2. Expert talks

Our research aims to develop a solution that addresses the identified problem for academic and industrial use. After conducting a preliminary RR, we interview industry experts to identify needs, challenges, and current practices. Through these interviews, we determine the relevance of the scheduling and allocation problem in the automotive industry, understand existing methods and influencing factors, assess the importance of optimization in practice, and identify limitations of current approaches.

Additionally, we conduct repeated expert talks during the framework's development to ensure that the experts' needs are addressed.

3.2.1. Interview questions

Based on the PICOC criteria and the DEC, we formulate the Interview Questions (IQs) in Table 3 to link academic know-how with practical insights on the topic.

3.2.2. Experts

We interview five different experts from different domains. We consider to interview one person from each of the three most influential domains driving change in automotive software architectures (see Mauser et al. (2025)): ADAS, powertrain, and infotainment. In addition, we include two experts working on the future HPC platform itself. Table 4 gives an overview about the experts, their domain, and years of experience.

3.2.3. Interview summary

Due to non-disclosure agreements, we cannot share the experts' detailed answers to the interview questions. We summarize the outcome of the expert talks in the following and conclude with the identified needs in the industrial context.

The expert interviews reveal that, in practice, task allocation is predominantly manual and domain-specific, with significant variation in abstraction level and implementation readiness. Certain experts use basic manual strategies based on task function, priority, or real-time constraints. Others remain in the conceptual or early deployment phases and are expected to transition towards more dynamic and automated approaches. In most cases, task-to-core allocation is driven by system-level requirements, such as safety, timing accuracy, processor type, memory usage, execution cycles, and task dependencies. KPIs that influence allocation decisions include runtime constraints, memory limits, CPU load, latency, and overall system stability. However, optimization is generally considered secondary to meeting stability and performance requirements. Reported challenges include reliance on static allocation, rigid prioritization schemes, memory constraints, and a lack of automation in scheduling and monitoring. In general, the findings underscore the need for research on dynamic scheduling, automated task distribution, and standardized frameworks for managing complex, resource-constrained automotive systems.

4. Related work

This section reviews the related work identified in the RR. Along with the industrial needs identified during the expert talks, we derive the detailed research gap and position our research. This approach allows us to elaborate the framework in Section 5.

The allocation of automotive software tasks in multi-core platforms has been widely studied. Numerous approaches have been proposed to improve efficiency, minimize overhead, and maintain system reliability. Existing solutions generally fall into static or dynamic allocation (see Section 2). In addition, several studies have explored optimization-based strategies to further enhance performance and resource utilization.

Table 3
Interview Questions (IQs).

ID	IQ
1	Do you use any specific allocation method?
2	How do you currently allocate tasks to cores?
3	What Key Performance Indicators (KPIs) or system properties influence your allocation decisions?
4	How do you evaluate whether an allocation is optimal?
5	What are the current pain points that our research can help address?

Table 4
Overview of experts and expertise.

ID	Expert role	Expert domain	Years of experience
1	SW architect	HPC platform	14
2	SW architect	ADAS	26
3	SW architect	Entire vehicle	20
4	SW architect	Powertrain	14
5	SW architect	Infotainment	13

4.1. Static approaches

Gupta et al. (2019) employ a Dependency Structure Matrix (DSM) to statically allocate runnables on a homogeneous dual-core microcontroller (NXP MPC5748G). The method seeks to minimize inter-core communication by allocating data-dependent runnables to the same core. However, the definition of these dependencies is not explicitly stated, and constructing the DSM involves substantial manual effort. Furthermore, in multi-core platforms, the system must still handle synchronization and inter-core communication, both of which introduce greater overhead compared to single-core architectures.

McLean et al. (2022) propose a static allocation method based on a greedy algorithm. This method offers a fast and straightforward solution that respects processor affinity and achieves balanced core utilization. However, since greedy algorithms prioritize local optima, this approach may result in suboptimal global utilization. Additionally, the method does not explicitly consider task dependencies or criticality, which limits its application to safety-critical systems.

Another notable contribution is presented by Hammond et al. (2015), who propose a static allocation method based on single-rate data flow (SRDF) graphs. The system is modeled as an SRDF graph and scheduled using a throughput-aware iterative algorithm (Moreira and Corporaal, 2014). The system's deterministic nature ensures consistent timing and repeatable behavior, making it well-suited for safety-critical applications. However, modeling systems as SRDF graphs is complex and error-prone, and the iterative scheduling incurs high computational costs. Furthermore, rollbacks are necessary when throughput constraints are violated, which increases overhead.

Due to the fixed nature of static methods, dynamic allocation has been increasingly explored in the literature.

4.2. Dynamic approaches

Rettkowski et al. (2015) propose two dynamic allocation methods. The first uses a First-Come, First-Serve (FCFS) strategy, in which tasks are queued in arrival order and executed sequentially as cores become available. This simple, fair, and predictable approach incurs low overhead but lacks prioritization and flexibility. Moreover, a single blocked task can delay all subsequent tasks in the queue. To overcome these limitations, they introduced a priority-based method that assigns tasks priority levels, enabling high-priority tasks to preempt lower-priority ones. While improving real-time responsiveness and suitability for safety-critical applications, this approach may cause starvation of non-critical tasks, involves subjective priority assignment, and increases debugging complexity.

Some studies have addressed the allocation problem as a multi-objective optimization challenge. For instance, McLean et al. (2022) aim to minimize priority-weighted chain latency by employing Simulated Annealing (SA) to efficiently explore feasible solutions and Genetic Algorithms (GA) for larger Time-Sensitive Networking (TSN) based test cases. While these approaches can improve mapping efficiency and reduce latency, their high computational complexity limits practicality in real-time automotive systems, where deterministic guarantees are essential.

4.3. Research gap

The analysis of the literature identified in the RR indicates that no single allocation strategy fully reconciles the trade-offs between efficiency, flexibility, and safety-critical requirements. Static methods offer determinism, but lack adaptability and often involve high modeling or computational costs. Dynamic methods improve adaptability to task availability at runtime, but may reduce predictability of execution order and increase variability in task waiting times. Although optimization-based approaches can be powerful, their computational demands often hinder their feasibility in real-time systems. These limitations along with the industrial needs identified in the expert talks highlight the need for methods that:

- combine the determinism of static methods with the adaptability of dynamic methods,
- explicitly consider task dependencies and priority attributes,
- and provide practical selection guidelines based on system metrics such as core utilization, throughput, task latency, and scheduling overhead.

The following section elaborates on the proposed framework based on the identified research gap and industry needs.

5. Framework

The scheduling and allocation of tasks in embedded automotive systems is still largely performed using manual and static strategies. While such approaches offer predictability, they struggle to scale with the increasing complexity of modern multi-core SoC platforms. To address this challenge, we propose the design-space exploration framework RunSoC that integrates automation and adaptability to analyze scheduling and allocation strategies under controlled and simplified assumptions. The positioning of RunSoC with respect to existing automotive scheduling and allocation frameworks is discussed in Section 7.1.

5.1. Framework design

The framework supports both scheduling and allocation decisions. At scheduling level, it enables switching between FCFS and PAS depending on objectives such as latency, throughput, and priority attributes. At allocation level, it combines static precomputation of core usage with dynamic provisioning to maximize utilization.

The input to the framework is a user-defined task set enriched with metadata, that include execution time, priority, and inter-task dependencies. Based on this information, the system constructs a directed flow

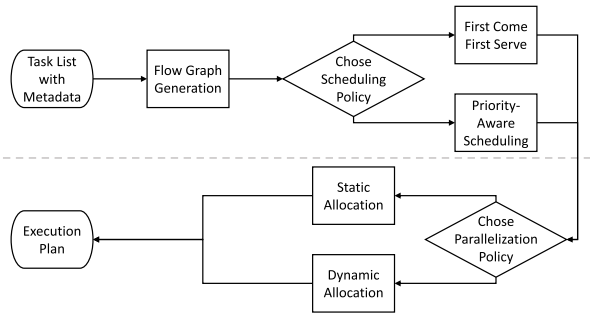


Fig. 2. Methodology overview.

graph that serves as the computational model for subsequent processing. The framework is organized into three main steps (see Fig. 2). These steps are presented in detail in the following.

The notation table (see Table 5) summarizes the symbol set utilized by the framework's core simulation logic and the underlying algorithms. Lowercase symbols denote task-level quantities (such as execution time), uppercase symbols denote system-level temporal bounds or parameters, and subscripts indicate semantic roles. The subsequent subsections will elaborate on the derivation of the symbols and provide further details.

5.2. Step 1: Task definition/flow graph generation

Users provide a task set $\Theta = \{\theta_1, \theta_2, \ldots, \theta_n\}$ with metadata including

- execution time $t_i \in \mathbb{R}^+$,
- task type (periodic tasks Θ_p for execution at fixed intervals determined by their period, denoted by T_i, or event-driven tasks Θ_e initiated upon satisfaction of their dependency constraints),
- priority $p_i \in \{0, 1, 2, 3, 4\}$, where the numbers represent increasing priority to reach lower latencies,
- and the dependency set Δ_i, where i indicates the i-th task in the set Θ. The symbol $\Delta_i = \{\theta_1, \theta_2, \ldots, \theta_m\}$ denotes the set of tasks that θ_i depends on for execution. In the context of periodic tasks, this set is found to be empty.

Task execution times are treated as deterministic upper-bound estimates provided by the system designer or software architect. The framework does not distinguish between worst-case or average execution times, as the primary objective is an agnostic, comparative architectural analysis rather than schedulability guarantees. Also, task priority is used solely as an ordering attribute within the scheduling policy and does not represent certification-relevant safety guarantees. It specifies the relative ordering among eligible event-driven tasks during scheduling and is intentionally decoupled from Automotive Safety Integrity Levels (ASILs) and hard real-time guarantees. In particular, a higher priority does not imply stricter latency requirements nor guaranteed execution. Event-driven tasks may therefore experience unbounded latency or may not be released at all if their dependency chain does not complete within the simulation horizon.

The input is transformed into a directed flow graph $G = (\Theta, E)$ by the flow graph generation module. The nodes Θ correspond to tasks $\theta_i \in \Theta$ and the directed edges E represent precedence constraints. Task θ_i depends on θ_j and can only be executed once all tasks in the set Δ_i are completed. The set of directed edges is defined as $E = \{\theta_j, \theta_i | \theta_j \in \Delta_i\}$.

The resulting graph G represents the definitive computational model of the system's execution structure. Automated construction enables task scheduling and partitioning while it ensures consistency with project specifications and reducing manual errors.

Table 5
Notation table of symbol set utilized by the methodology.

Notation	Meaning
Task definition/flow graph generation	
Θ	Set of tasks
t_i	Execution time of task θ_i
p_i	Priority of task θ_i
Θ_p/Θ_e	Task type: periodic vs. event-driven
T_i	Period of periodic tasks $\Theta_{p,i}$
Δ_i	Dependency set of task θ_i
E	Directed edges set
G	Directed flow graph based on Θ and E
Task scheduling	
α	Scheduling policy: FCFS vs. PAS
Q_{fcfs}	Scheduling queue FCFS
Q_{pas}	Scheduling queue PAS
s_i	Starting time of task θ_i
f_i	Finishing time of task θ_i
c_j	Allocated core of task θ_i
C	Number of cores
(τ)	Current simulation time
$\phi_{i,j}$	Earliest next periodic activation time of task θ_i on core c_j
η_i	Eligible time of task θ_i
id_i	Unique identifier of task θ_i
O	Computational cost
K	Number of tasks examined per core
Parallelization	
β	Allocation policy: Static vs. dynamic
$P(\tau)$	Instantaneous parallelism
$P_{\max}$	Maximum parallelism
W	Total work: Sum of all task execution times as time-valued workload metric
L_i	Longest path leading to specific task θ_i
T_{cp}	Critical/Maximum path length in DAGs
T_{work}	Work-based lower bound: ideal make-span as total work divided by cores
T_{total}	Observed makespan: actual completion time based on scheduling and allocation
C^*	Theoretical optimal core count
f_{ser}	Serial fraction
f_{par}	Parallel fraction
ϵ	Targeted fraction of maximum speedup
$N_{\min}$	Minimum cores for target speedup
C_{alloc}	Actual core count assignment
$C_{alloc,(\tau)}$	Dynamic core count assignment
Simulation time estimation	
$T_{\max}$	Simulation time
I	Multiple of total work

5.3. Step 2: Task scheduling

After construction of the graph $G = (\Theta, E)$, tasks are scheduled to determine execution order. We assume a homogeneous multi-core processor with $C \in \mathbb{N}$ identical cores (details in Section 5.4) and non-preemptive execution. This means that, once started on a core, a task runs to completion.

The schedule is defined as $\{(s_i, f_i, c_j)\}_{i=1}^{|\Theta|}$, where $s_i \geq 0$ is the start time, $f_i = s_i + t_i$ the finish time, and $c_j \in \{1, \ldots, C\}$ the core allocation, such that:

- **Dependency:** Event-driven tasks start only after all dependencies complete: $s_i \geq \max_{\theta_j \in \Delta_i} f_j$.
- **Periodicity:** Periodic tasks activate at integer multiples of T_i: $s_i = k \cdot T_i, \forall \theta_i \in \Theta_p, k \in \mathbb{N}$.
- **Exclusive cores:** No core executes two tasks simultaneously: if $c_j = c_k$ and $i \neq k$, $[s_i, f_i) \cap [s_k, f_k) = \emptyset$.

At any time $\tau \geq 0$, the subsequent sets can be retrieved:

- **Running** $(\tau) \subseteq \Theta$**:** Tasks executing at τ.
- **Completed** $(\tau) \subseteq \Theta$**:** Tasks with $f_i < \tau$.
- **Eligible** $(\tau) \subseteq \Theta$**:** Event-driven tasks with all dependencies completed, and periodic tasks where τ is a multiple of T_i, excluding

Completed and **Running**: **Eligible**$(\tau) = \{\theta_i \in \Theta_e \mid \Delta_i \subseteq$ **Completed**$(\tau)\} \cup \{\theta_j \in \Theta_p \mid \tau \pmod{T_j} = 0\} \setminus ($**Completed**$(\tau) \cup$ **Running**$(\tau))$.

- **IdleCores**$(\tau) \subseteq \{1, \ldots, C\}$**:** Cores not executing a task, sorted ascending index order.

The eligible time η_i for each task θ_i is determined by:

$$\eta_i = \begin{cases} \max_{\theta_j \in \Delta_i} f_j, & \theta_i \in \Theta_e, \\ k \cdot T_i, & \theta_i \in \Theta_p. \end{cases}$$

The scheduler is event-driven. Decision-making is initiated at defined *decision points*:

- at $\tau = 0$ (initialization),
- on task completion: $\tau \in \{f_i \mid \theta_i \in \Theta\}$,
- or at periodic activations: $\tau \in \{k \cdot T_i \mid k \in \mathbb{N},\ \theta_i \in \Theta\}$.

Between decisions, states are static. At each decision point, the following actions can be taken:

- Mark newly finished tasks **Completed**.
- Add new **Eligible** tasks and recalculate $s_i \forall \theta_i \in$ **Eligible**.
- While **Eligible** and **IdleCores** are non-empty, dispatch tasks to idle cores according to chosen policy, and change the state of corresponding task to **Running**.

This work-conserving approach never leaves a core idle if tasks are ready to be executed.

5.3.1. Strict periodicity guard

To maintain strict periodicity, event-driven tasks must not delay an imminent periodic activation. For each core $c_j \in \{1, \ldots, C\}$, $\phi_{i,j}$ denotes the earliest next activation time of any periodic task $\theta_i \in \Theta_p$, with $\phi_{i,j} = s_i + T_i$ by default. The concept of strict periodicity implies $s_i = \eta_i$.

A task θ_i is safe to dispatch on c_j at time τ iff:

$$\textbf{Safe}(c_j, \theta_i, \tau) := (\theta_i \in \Theta_p) \vee (\tau + t_i | \forall \theta_i \in \Theta_e) \leq \phi_{i,j}. \tag{3}$$

Dispatch is only allowed if the task is periodic or if an event-driven task will complete before the next guaranteed periodic activation $\phi_{i,j}$.

5.3.2. Scheduling algorithms

Efficient task allocation in multi-core systems relies on scheduling algorithms that determine the execution order of ready tasks. Different algorithms exhibit distinct trade-offs in core utilization, task waiting time, and completion latency. We consider two representative approaches: FCFS, that prioritizes simplicity and preserves arrival-order execution, and PAS, which orders eligible tasks based on designer-assigned priority attributes. These policies serve as the basis for the evaluation of alternative allocation strategies in automotive embedded systems.

The **FCFS policy** orders eligible tasks by eligible time η_i, with ties broken by task ID $id(\theta_i)$:

$$Q_{\text{fcfs}}(\tau) = \textbf{Eligible}(\tau) \text{ ordered by } (\eta_i, \text{id}(\theta_i)). \tag{4}$$

Tasks with smaller IDs precede in case of equal η_i.

At every decision point, the **dispatch rules** are applied to select the next task for execution: Take the first task θ_i from Q_{fcfs}; Periodic-first admission: If $\theta_i \in \Theta_p$ and $\eta_i = \tau$ with **IdleCores** non-empty, allocate it to the first idle core immediately, and set $\eta_i \leftarrow \tau + T_i$; If $\theta_i \in \Theta_e$, allocate to the lowest-index idle core if **Safe** per (3). Otherwise, bypass to the next task. Stop when no safe allocations or no idle cores remain.

This approach guarantees strict periodicity as event-driven tasks never block imminent periodic activations.

Properties: Tasks run in eligibility order from satisfied event-driven dependencies or periodic triggers (dependency- and time-driven); Earlier-eligible tasks are never overtaken, with the periodicity guard blocking unsafe dispatches (fair in arrival time); With fixed t_i, η_i, C, and id(θ_i), FCFS yields the same sequence (deterministic); In a finite non-preemptive, work-conserving DAG, all ready tasks eventually start (no starvation); Ignores p_i so urgent tasks may wait, and periodicity enforcement can bypass event-driven tasks, causing idle cores until the next periodic activation and reduced utilization (limitation).

Computational cost: The total scheduling overhead is given as $O(|\Theta| \log |\Theta| + |E| + C \log K)$, with: $|\Theta|$ total number of tasks; $|E|$ total number of edges in the DAG; $O(|\Theta| \log |\Theta|)$ ready queue management. Each task is inserted once into a min-heap, giving $|\Theta|$ insertions at cost $\log |\Theta|$ each; $O(|E|)$ update of eligible times. When a task completes, all dependent tasks (outgoing edges) are notified. Total cost proportional to $|E|$; $O(C \log K)$ strict periodicity guard checks. At each decision point, safe tasks from the ready queue are allocated to up to C cores. Unsafe tasks are bypassed until a valid one is found; K is the number of tasks examined per core. Extracting from the min-heap costs $\log K$ per check.

The **PAS policy** schedules higher p_i first and uses FCFS for equal p_i:

$$Q_{\text{pas}}(\tau) = \textbf{Eligible}(\tau) \text{ ordered by } (-p_i, \eta_i, \text{id}(\theta_i)), \tag{5}$$

where primary key $-p_i$ orders by priority with highest priority tasks at front; Secondary key η_i applies FCFS within same priority groups; Tertiary key id(θ_i) resolves remaining ties in priority p_i and eligible time η_i.

Dispatch rules: As in FCFS, enforce strict periodicity and periodic-first admission; Repeatedly take the head of Q_{pas} and allocate it to the lowest-index idle core c_j if **Safe** (3); Bypass unsafe tasks until no safe allocations or no idle cores remain.

Properties: Always schedules highest-priority eligible tasks first (responsiveness to urgency); Equal-priority tasks keep FCFS order to preserve arrival-order execution (FCFS tie-breaking); Promoting high-priority tasks can delay lower-priority ones without reducing total project duration (trade-off).

Computational cost: The total scheduling overhead is given as $O(|\Theta| \log |\Theta| + |E| + C \log K)$, identical to FCFS.

5.4. Step 3: Parallelization

Scheduling with FCFS or PAS determines task order, but execution on a single core limits performance. Modern multi-core systems can run independent tasks concurrently, reducing the total completion time (makespan). Task to core allocation aims to minimize makespan while satisfying dependencies and scheduling constraints. We consider two approaches: Static core allocation (offline, fixed before execution) and dynamic core allocation (online, adapting at runtime).

At a given time τ, we investigate the following **key metrics:** Instantaneous parallelism $P(\tau) = |$**Eligible**$(\tau)|$; Maximum parallelism $P_{\max} = \max_{\tau \geq 0} P(\tau)$; Critical path length to a task θ_i $L_i = t_i + \max_{\theta_j \in \Delta_i} L_j$; Critical path length in the DAG $T_{cp} = \max_{\theta_i \in \Theta} L_i$ (minimum possible makespan); Total work $W = \sum_{i=1}^{|\Theta|} t_i$; Work bound for C cores $T_{work} = W/C$; Makespan bound $T_{total} \geq \max(T_{cp}, T_{work})$.

Core allocation problem: The objective of the core allocation problem is to ascertain the optimal number of processing cores, denoted by C^*, that would minimize the makespan while ensuring the efficient utilization of resources. The optimal core count C^* is reached when adding more cores yields no speedup:

$$C^* = \min\{C \mid T_{total} \approx T_{cp}\}. \tag{6}$$

Static and dynamic strategies differ in how the optimal number of cores C^* is determined and updated.

5.4.1. Static core allocation

Offline strategy that uses known DAG structures and task times. Given the critical path length T_{cp} and total work W, serial fraction is $f_{ser} = T_{cp}/W$ while parallel fraction is $f_{par} = 1 - f_{ser}$. Amdahl's law gives the minimal core count for a speedup fraction ε of maximum speedup:

$$N_{\min} = \left\lceil \frac{\varepsilon \cdot f_{par}}{f_{ser}(1 - \varepsilon)} \right\rceil. \tag{7}$$

Constraints: (1) $C > P_{\max}$ – utilization of additional cores remains constrained to Pmax; (2) $C > N_{\min}$ – utilization of cores beyond $N_{\min}$ give negligible gain.

The core allocation strategy determines the optimal number of cores C_{alloc} to allocate to the scheduler to avoid resource waste by ensuring that only effectively usable cores are allocated.

$$C_{\text{alloc}} = \min(C, P_{\max}, N_{\min}). \tag{8}$$

Execution proceeds by initializing $\tau = 0$, selecting up to C_{alloc} eligible tasks, dispatching them to lowest-index idle cores, advancing τ to the earliest finish time among all tasks in **Running**, updating sets **Completed**, **IdleCores**, and **Eligible**.

5.4.2. Dynamic core allocation

Online strategy that adapts $c_{\text{alloc}}(\tau)$ to system state at runtime. At each decision point τ,

$$c_{\text{alloc}}(\tau) = \min(|\textbf{IdleCores}|, |\textbf{Eligible}(\tau)|), \tag{9}$$

to ensure that cores are only allocated if tasks are ready to execute. Dispatch and update steps mirror the static method, and thus replace C_{alloc} with $c_{\text{alloc}}(\tau)$.

5.5. Simulation time estimation

To assess long-term effects of mixed event-driven and periodic tasks, simulations run multiple DAG repetitions until a sufficient amount of work, W, has elapsed. With I to denote a multiple of W, we define the simulation time as $T_{\max} = I \cdot W$.

The final output of the framework is a concrete execution plan that specifies the allocation of tasks to cores and their exact execution sequence. This plan is visualized in the form of Gantt charts to provide a clear temporal overview of resource utilization. The framework is domain-agnostic, operating on abstract task metadata, and is designed modular to facilitate integration of new scheduling heuristics or allocation policies.

5.6. Framework implementation

While the design phase establishes a general model for scheduling and parallelization, this section presents the concrete implementation into a unified, executable framework. For clarity and reproducibility, Table 5 summarizes the notation used throughout this section. The framework enables reproducible scheduling experiments through consolidation of periodic and event-driven semantics, task selection policies, and core allocation strategies into a coherent workflow.

The framework consists of:

- a **Next.js frontend** for task input and DAG visualization,
- a **Python backend** that implements the scheduling, allocation, and dispatch logic described in this section,
- and an **Nginx deployment** to provide seamless communication between frontend and backend, creating a unified user platform.

The source code is publicly available on GitHub and Zenodo to enable researchers and practitioners to fork, adapt, and extend it.

5.6.1. Framework execution model and main loop

The prototype implements three core stages that correspond to the algorithms presented in this section:

1. **DAG construction**: User-defined tasks and dependencies are converted into the workload graph $G = (\Theta, E)$.
2. **Scheduling**: An event-driven scheduler orders eligible tasks according to the selected scheduling policy $\alpha \in \{\text{FCFS}, \text{PAS}\}$.
3. **Parallelization**: Core allocation is determined either statically at simulation start or dynamically at runtime, as specified by the allocation policy $\beta \in \{\text{static}, \text{dynamic}\}$.

The design enables deterministic evaluation of *makespan*, *resource utilization*, and related performance metrics in a controlled simulation environment. The implementation assumes a non-preemptive execution model on homogeneous cores and excludes communication overhead in order to isolate scheduling and allocation effects.

The simulation flow is driven by a global event loop that advances simulation time from one decision point to the next. Decision points occur at initialization, task completions, and periodic activations. At each decision point, the scheduler applies the selected scheduling policy α to order the eligible task set, and the allocator applies the selected allocation policy β to determine the number of available cores. Tasks are then dispatched before the simulation time advances to the next event.

Algorithm 1 defines the main scheduling loop and serves as the top-level control structure of the simulation. Its inputs comprise the workload graph $G = (\Theta, E)$, task execution times t_i, the number of available cores C, the simulation horizon parameter I, and the configuration parameters α and β. Parameter α specifies the task scheduling policy (FCFS or PAS), while parameter β selects the core allocation strategy (static or dynamic). The algorithm iteratively advances simulation time, invokes the selected scheduling and allocation submodules, and dispatches tasks to cores. Its output is a complete execution schedule over the simulation horizon $T_{\max}$, represented by task start times, finish times, and core assignments.

Algorithm 1 Main scheduling algorithm (main_scheduler).

Input: $G = (\Theta, E)$, t_i, C, I, α, β
Initialize $\tau \leftarrow 0$; **Completed** $\leftarrow \emptyset$; **Running** $\leftarrow \emptyset$
Eligible $\leftarrow \{\theta_i \in \Theta_e \mid \Delta_i = \emptyset\} \cup \{\theta_i \in \Theta_p\}$; **IdleCores** $\leftarrow \{1, 2, \ldots, C\}$
Compute $P_{\max}, N_{\min}, W$
Compute $T_{\max} = I \cdot W$
if β = static **then**
  (C_{alloc}, **AvailableCores**) $\leftarrow$ `static_core_allocation`($P_{\max}, N_{\min}, C$)
end if
for $\tau < T_{\max}$ **do**
  Eligible $\leftarrow \alpha$(**Eligible**) {apply FCFS or PAS scheduling}
  if β = dynamic **then**
    (C_{alloc}, **AvailableCores**) $\leftarrow$ `dynamic_core_allocation`(τ, **Eligible**, **IdleCores**)
  end if
  for all $\theta_i \in$ **Eligible do**
    Ready $\leftarrow$ first C_{alloc} tasks in **Eligible**
    for all $\theta_i \in$ **Ready do**
      Move θ_i to **Running**; remove θ_i from **Eligible**; remove θ_i from **Ready**
      $(s_i, f_i, c_j) \leftarrow$ dispatch (θ_i, τ, **AvailableCores**, **Eligible**)
      Remove c_j from **IdleCores**; Remove c_j from **AvailableCores**
    end for
    $\tau \leftarrow \min\left(\{f_i : \theta_i \in \textbf{Running}\} \cup \{\eta_j : \theta_j \in \Theta_p\}\right)$
    for all $\theta_i \in$ **Running** with $f_i = \tau$ **do**
      Move θ_i to **Completed**; release c_j back to **IdleCores** and **AvailableCores**
      for all successor r^s of θ_i **do**
        Add r^s to **Eligible**; set $\eta_s \leftarrow \tau$
      end for
    end for
  end for
  Record schedule $\{(s_i, f_i, c_j)\}_{i=1}^{|\Theta|}$ for iteration k
end for
Output: Allocation and scheduling across $T_{\max}$

5.6.2. Task scheduling

Task scheduling determines the execution order of tasks that are eligible at a given simulation time. Both scheduling algorithms take as input the set **Eligible** of ready tasks and produce an ordered ready queue Q_{ready}. The ordering policy is selected via parameter α and remains fixed for the duration of the simulation. FCFS sorts tasks by their eligible start time. PAS sorts first by priority, followed by eligible time. Algorithm 2 details the FCFS ordering. Algorithm 3 details PAS ordering.

Algorithm 2 FCFS (fcfs).

Input: **Eligible**
Assign $Q_{ready} \leftarrow \{\, \theta_i \in \textbf{Eligible} \mid \text{ordered by } (\eta_i, \mathrm{id}_i) \,\}$
Output: Q_{ready}

Algorithm 3 PAS (pas).

Input: **Eligible**
Assign $Q_{ready} \leftarrow \{\, \theta_i \in \textbf{Eligible} \mid \text{ordered by}(-p_i, \eta_i, \mathrm{id}_i)$
Assign $r^* \leftarrow \min_{\theta_i \in Q_{ready}}(-p_i, \eta_i, \mathrm{id}_i)$
Output: Q_{ready}

5.6.3. Core allocation

Core allocation determines how many processing cores are made available for task execution at each decision point. The allocation strategy is selected via parameter β. Static allocation computes the number of usable cores once prior to execution, while dynamic allocation adapts the number of allocated cores at runtime based on the current system state. Both allocation algorithms explicitly return the number of allocatable cores and the corresponding set of available cores used by the dispatcher. Algorithm 4 details static allocation. Algorithm 5 details dynamic allocation.

Algorithm 4 Static core allocation (static_core_allocation).

Input: $P_{\max}$, $N_{\min}$, C
Compute $C_{\text{alloc}} \leftarrow \min(C, P_{\max}, N_{\min})$
Initialize **IdleCores** $\leftarrow \{1, 2, \ldots, C_{\text{alloc}}\}$
Output: C_{alloc}, **IdleCores**

Algorithm 5 Dynamic core allocation (dynamic_core_allocation).

Input: τ, **Eligible**(τ), **IdleCores**(τ)
Compute $c_{\text{alloc}}(\tau) \leftarrow \min\left(|\textbf{IdleCores}(\tau)|, |\textbf{Eligible}(\tau)|\right)$
Allocate **AvailableCores**$(\tau) \leftarrow \{c_j \in \textbf{IdleCores}(\tau) | 1 \le j \le c_{\text{alloc}}(\tau)\}$
Output: $c_{\text{alloc}}(\tau)$, **AvailableCores**

5.6.4. Dispatch rule

The dispatcher implements the framework's dispatch rule, i.e., the mapping of a selected ready task θ_i to a concrete core at the current decision time τ. It takes as input the current simulation time, the set of currently idle cores, and the task θ_i selected from Q_{ready}. As a result, it returns the start time s_i, finish time f_i, and assigned core c_j. The dispatcher follows a *periodic-first* admission policy: if θ_i is periodic and released at τ, it is immediately assigned to the lowest-index idle core to preserve strict periodicity. After dispatching a periodic task, the dispatcher computes its next activation time and reinserts the corresponding release into the ready structure such that the periodic task becomes eligible again at $\tau + T_i$. Algorithm 6 details the dispatch logic.

Algorithm 6 Dispatcher.

Input: θ_i, τ, **IdleCores**, Q_{ready}
if $\theta_i \in \Theta_p \wedge \eta_i = \tau \wedge \textbf{IdleCores} \neq \emptyset$ **then**
  $c \leftarrow \min(\textbf{IdleCores})$
  Allocate θ_i to c
  Remove c from **IdleCores**
  Compute $\phi_{i,j} = \tau + T_i$ {next activation time}
  Insert $(\theta_i, \phi_{i,j})$ into Q_{ready}
else
  for $c \in$ **IdleCores do**
    if `strict_periodicity_guard`$(c, \theta_i, \tau, \phi_{i,j}) =$ **true then**
      Allocate θ_i to c
      Remove c from **IdleCores**
      Compute $s_i = \tau$
      Compute $f_i = \tau + t_i$
      break
    else
      Compute $s_i = \phi_{i,j} + t_x$ {t_x is the execution time of the next activated periodic task}
    end if
  end for
end if
Output: (s_i, f_i, c_j)

For event-driven tasks, the dispatcher invokes the strict periodicity guard before assigning a core. The guard uses the next guaranteed periodic activation time $\phi_{i,j}$ on a candidate core c_j as a safety boundary and admits θ_i only if executing it starting at τ completes before $\phi_{i,j}$. Otherwise, the task is not started at the current decision point. Operationally, the dispatcher iterates over the available idle cores in ascending order and selects the first core on which the guard evaluates to true. It then fixes the task interval to $[s_i, f_i) = [\tau, \tau + t_i)$ and returns the assignment. If no idle core satisfies the guard, θ_i remains pending until a later decision point, ensuring that no event-driven execution can delay a periodic release under the non-preemptive execution model assumed by the simulator. Algorithm 7 details the strict periodicity guard.

Algorithm 7 Strict periodicity guard (strict_periodicity_guard).

Input: c, θ_i, τ, $\phi_{i,j}$
Allocate **Safe** $\leftarrow (\theta_i \in \Theta_p) \wedge (\tau + t_i \le \phi_{i,j})$
Output: **Safe**

6. Results & discussion

This section evaluates the framework using two representative ADAS task sets. Execution time, type, priority, and period are same for both cases. The sole difference lies in their dependency structures. Use case 1 features a long and dominant critical path, representing highly sequential workloads with low parallelization potential. Use case 2 presents a balanced structure with multiple independent branches, enabling greater concurrency and core utilization. These contrasting scenarios allow assessment of the scheduler and allocation strategies under both parallelism-constrained and balanced workloads. Details on the task sets and the related DAGs are publicly available on GitHub and Zenodo.

6.1. Use case descriptions

Use Case 1 – Long critical path: Twenty tasks, primarily event-driven, arranged in a deep dependency chain, form a DAG with a pronounced sequential segment. Parallelism is constrained irrespective of the core count. **Use Case 2 – Balanced structure:** Twenty tasks with

dependencies distributed across several branches produce a more balanced DAG that facilitates simultaneous execution. **Analytical metrics** to quantify the parallelization potential of both DAGs include total work W, critical path length T_{cp}, maximum parallelism $P_{\max}$, and minimal speedup core count $N_{\min}$ (with $\varepsilon = 0.9$). Table 6 summarizes these characteristics.

Table 6
Characteristic values for the two use cases.

Use case	W (ms)	T_{cp} (ms)	$P_{\max}$	$N_{\min}$
1	595	575	2	1
2	595	270	4	11

6.2. Simulation setup

The simulation setup considers six homogeneous cores and a simulation time of $T_{\max} = 3 \cdot W = 1785$ ms. The performance is evaluated using: Executed tasks (total and per-core counts); core utilization (per-core and average utilization percentages); latency (average time tasks spend queued before dispatch). All evaluated configurations assume non-preemptive execution. Task migration between cores is permitted, and associated cache-related overheads are not modeled. This abstraction allows isolating scheduling and allocation effects, but may overestimates achievable performance in systems where migration is restricted.

6.3. Static vs. dynamic core allocation

6.3.1. Use case 1: Long critical path

Static allocation utilizes only one core with $C_{\text{alloc}} = 1$ (see Eq. (8)), whereas dynamic allocation utilizes all six available cores (see Eq. (9)). Simulation runtime differs negligibly (15 ms, 0.84 %), confirming that the observed effects stem from allocation behavior rather than simulation overhead. Figs. 3 and 4 illustrate the resulting schedules as Gantt charts.

Core utilization: Static allocation achieves near-saturation on its single active core (91.81 %) but leaves the remaining five cores idle. Consequently, its average utilization drops to 15.3 %. In contrast, dynamic allocation distributes work across all cores, achieving an average utilization of 56.35 % (see Tables 7 and 8). The descending utilization pattern by core index reflects the deterministic selection of the lowest-indexed idle cores first. This highlights that dynamic allocation improves utilization not by increasing the intrinsic parallelism of the workload, but by exploiting already available task-level parallelism whenever dependencies permit execution.

Table 7
Core utilization (static) for use case 1.

Core	Exec. time (ms)	Utilization (%)
0	1625	91.81
1–5	0	0
Average	270.83	15.3

Table 8
Core utilization (dynamic) for use case 1.

Core	Exec. time (ms)	Utilization (%)
0	1575	88.24
1	1080	60.50
2	985	55.18
3	885	49.58
4	840	47.06
5	670	37.54
Average	1005.83	56.35

Executed tasks: As shown in Table 9, dynamic allocation executes all 20 task types and completes 211 task instances (35.17 per core). Static allocation, in contrast, executes only 17 task types and 66 task instances in total. Runnables 18, 19, and 20 – located at the tail of the dependency graph – are not executed because their predecessor chains do not complete within the simulation horizon under single-core constraints. Importantly, these tasks are not actively skipped by the scheduler; rather, they are never released due to unmet dependencies under limited computational capacity. This behavior reveals an architectural bottleneck rather than a scheduling flaw and demonstrates how static allocation can mask infeasible workload configurations during early design.

Table 9
Executed task counts for use case 1.

Core alloc. method	N_{type}	N_{total}	N_{avg}	N_{active}
Static	17	66	11	66
Dynamic	20	211	35.17	35.17

Latency: As shown in Table 10, average per-task latency remains comparable between static (26.52 ms) and dynamic allocation (24.03 ms), despite the significantly higher throughput achieved dynamically. The higher total waiting time observed under dynamic allocation stems from increased task volume rather than degraded responsiveness. Additionally, a conservative global periodicity guard causes avoidable queuing effects, indicating optimization potential independent of the chosen allocation strategy.

Table 10
Latencies for use case 1.

Core alloc. method	Σ_{total} (ms)	Σ_{avg} (ms)	T_{avg} (ms)
Static	1750	26.52	24.62
Dynamic	5070	24.03	28.6

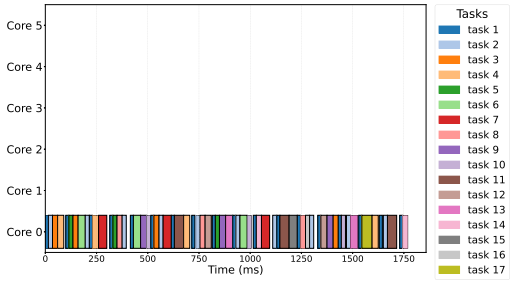


Fig. 3. Use case 1 - Gantt chart for static core allocation using FCFS.

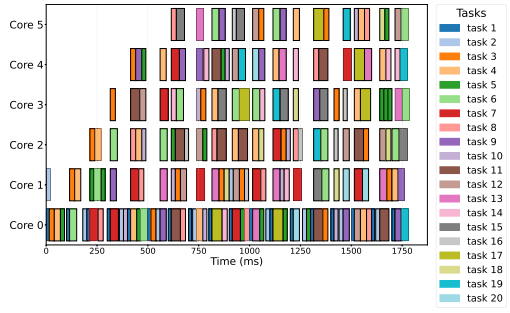


Fig. 4. Use case 1 - Gantt chart for dynamic core allocation using FCFS.

6.3.2. Use case 2: Balanced structure

Static allocation utilizes four cores with $C_{\text{alloc}} = 4$ (see Eq. (8)), while dynamic allocation again exploits all six cores (see Eq. (9)). Runtime

differences remain negligible (10 ms, 0.56 %). Figs. 5 and 6 present the corresponding Gantt charts.

Core utilization: Due to the more balanced dependency structure, both strategies achieve higher and more uniform utilization. Static allocation reaches an average utilization of 46.81 %, while dynamic allocation improves this further to 64.71 % (see Tables 11 and 12). Unlike Use Case 1, the balanced DAG enables static allocation to benefit from parallel execution, narrowing the utilization gap between the two approaches.

Table 11
Core utilization (static) for use case 2.

Core	Exec. time (ms)	Utilization (%)
0	1615	90.99
1	1160	65.35
2	1100	61.97
3	1110	62.54
4–5	0	0
Average	830.83	46.81

Table 12
Core utilization (dynamic) for use case 2.

Core	Exec. time (ms)	Utilization (%)
0	1615	90.48
1	1160	64.99
2	1175	65.83
3	1095	61.34
4	1000	56.02
5	885	49.58
Average	1155	64.71

Executed tasks: As shown in Table 13, both allocation strategies execute all 20 task types. Dynamic allocation achieves higher total throughput (240 vs. 178 tasks) by activating all available cores, whereas static allocation achieves slightly higher per-active-core throughput. This illustrates that static allocation can be competitive when workload parallelism aligns well with the chosen core count, but remains limited by unused cores when additional capacity is available.

Table 13
Executed task counts for use case 2.

Core alloc. method	N_{type}	N_{total}	N_{avg}	N_{active}
Static	20	178	29.67	44.5
Dynamic	20	240	40	40

Latency: As shown in Table 14, average latencies remain similar across both allocation strategies (29 ms - 34 ms) and close to the task execution times. This indicates a workload-intrinsic bottleneck in which tasks spend nearly as much time waiting as executing. The result suggests that further latency improvements require reducing idle waiting times caused by dependency structure or periodic activation constraints, rather than modifying the allocation strategy alone.

Table 14
Latencies for use case 2.

Core alloc. method	Σ_{total} (ms)	Σ_{avg} (ms)	T_{avg} (ms)
Static	6095	34.24	28.01
Dynamic	6940	28.92	28.88

6.4. Scheduler policy comparison: FCFS vs. PAS

Using 50 random mixed-structure task sets, average latency was compared across core counts (see Figs. 7 and 8). The primary difference

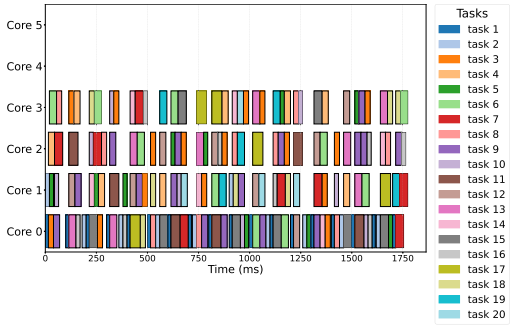


Fig. 5. Use case 2 – Gantt chart for static core allocation using FCFS.

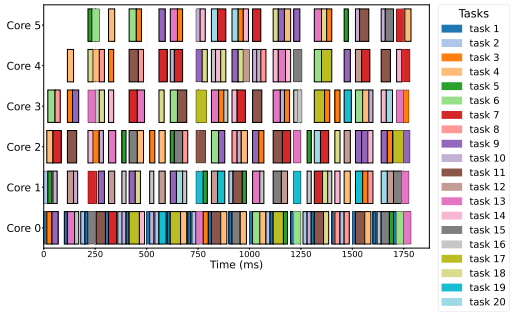


Fig. 6. Use case 2 – Gantt chart for dynamic core allocation using FCFS.

occurs at a single-core configuration: FCFS consistently outperforms PAS in this regime. At two cores, both policies show a slight increase in waiting time, likely due to distribution overhead. Beyond three cores, differences diminish, as idle cores mitigate low-priority blocking in PAS.

- **Core-limited platforms (1–2 cores):** FCFS is preferable due to its lower scheduling overhead and reduced blocking effects.
- **Multi-core platforms (≥ 3 cores):** PAS exhibits latencies comparable to FCFS while enabling deterministic ordering of eligible tasks based on priority attributes when sufficient cores are available.
- **Impact of allocation strategy:** Allocation policy has limited influence on average latency but significantly affects utilization and overall throughput.

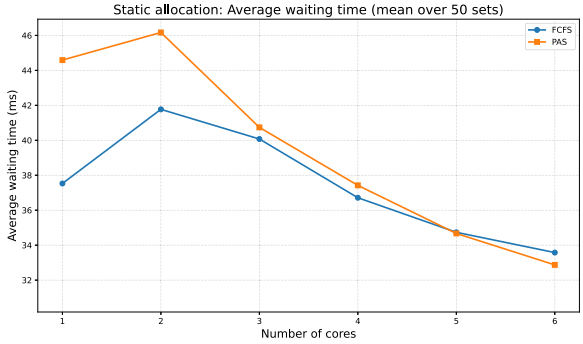


Fig. 7. Average latency for static core allocation FCFS vs. PAS.

7. Discussion

The evaluation results demonstrate that scheduling and allocation choices significantly influence system-level behavior even under simplified modeling assumptions. Static allocation provides strong predictability and high per-core utilization when resources are sufficient, but it may fail to execute all event-driven tasks when workload exceeds available

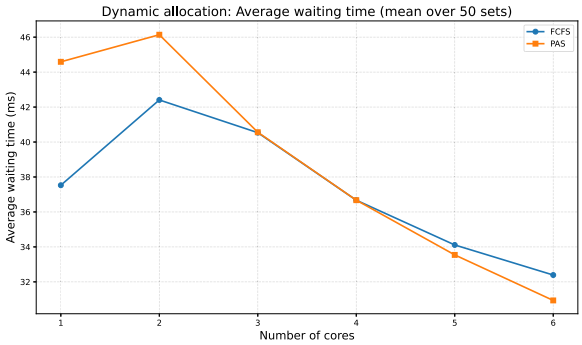


Fig. 8. Average latency for dynamic core allocation FCFS vs. PAS.

computational capacity. In such cases, tasks are not actively skipped by the scheduler but remain unreleased due to unmet dependency chains (see Use Case 1), indicating insufficient architectural resources rather than algorithmic failure.

Dynamic allocation, in contrast, improves overall utilization and throughput by adapting core usage at runtime (see Use Case 2). This flexibility allows more tasks to complete within the simulation horizon, at the cost of increased variability in task start times and reduced execution predictability. These observations highlight a fundamental trade-off between predictability and adaptability, which cannot be resolved by a single scheduling strategy but must be evaluated in the context of workload structure and architectural constraints.

The results further indicate that dynamic allocation is most beneficial for workloads with fluctuating parallelism, whereas static allocation remains competitive when dependency chains and periodic activations dominate execution. This reinforces the insight that allocation strategies should be selected based on workload structure and timing characteristics rather than core count alone.

Finally, the evaluation shows that scheduling policies influence task ordering and latency among eligible tasks but do not compensate for fundamentally insufficient resources. For example, priority-aware scheduling improves responsiveness within the eligible task set, yet cannot guarantee execution of all tasks when core availability is limited. This underscores the role of RunSoC as a design-space exploration framework that exposes architectural bottlenecks and trade-offs early in the system design process.

7.1. Positioning with respect to existing frameworks

RunSoC is not intended to replace or outperform existing automotive scheduling and allocation frameworks. Prior work such as Rettkowski et al. (2015) and Gupta et al. (2019) focuses on deriving deployable task-to-core allocations or runtime schedulers under specific assumptions, for example static DSM-based allocation or priority-driven execution. In contrast, RunSoC is explicitly designed as a design-space exploration framework. Its objective is to enable systematic comparison of alternative scheduling and allocation strategies under simplified and controllable assumptions, rather than to synthesize a production-ready schedule. For this reason, RunSoC emphasizes transparency of scheduling decisions, reproducibility of experiments, and exposure of architectural metrics such as utilization, throughput, and task latency. Due to these differing goals and modeling assumptions, a direct quantitative baseline comparison with existing automotive schedulers is not meaningful. Instead, RunSoC complements prior work by supporting early-stage architectural decision-making before detailed real-time and safety constraints are introduced.

8. Threats to validity

The previous section discussed the framework's capabilities and limitations, while the outlook section presents how to overcome these. Here, we focus on threats related to the RR and expert talks. Following the taxonomy by Ampatzoglou et al. (2019), we address three main categories: *threats to study selection*, *threats to data collection*, and *threats to research validity*.

Threats to study selection: Although this work followed an RR approach, its setup was closely aligned with that of an SLR. Both main authors and two independent researchers jointly defined and refined the review protocol. The search strategy, based on the guidelines of Mourão et al. (2020), enabled an efficient RR with high precision and recall. We relied on the high quality of the Scopus database. Snowballing was omitted due to the RR design, which may introduce publication bias.

The PICOC criteria, search string, and inclusion/exclusion criteria (see Table 1) were thoroughly discussed and piloted by the main authors, with arbitration by the independent researchers in case of disagreement. To bridge the gap between academia and industry, and mitigate the risk of outdated information, the RR was complemented by expert talks. Experts from relevant domains contributed diverse perspectives (see Table 4). While their strong professional experience may introduce bias through established viewpoints, this risk is further mitigated as discussed below. The inclusion of young graduates or professionals may provide further valuable approaches and perspectives.

Threats to data collection: We mitigated data extraction bias through commonly agreed DEC derived from the research question and PICOC criteria. A piloting phase ensured a common understanding by the two main authors. Although extraction was conducted by one author, the use of standardized DEC (see Table 2) minimized subjective interpretation. The same principle applied to the expert talks, which followed a consistent set of IQs (see Table 3) with a clear, structured yet open format to avoid leading or biased responses.

Threats to research validity: To minimize research bias, the two main authors jointly designed and refined the review protocol in accordance with established guidelines. The first author specializes in automotive software engineering, while the second focuses on general software engineering, reducing domain-specific bias. Two additional researchers, one with expertise in automotive functional safety and another in general software engineering, reviewed the protocol to ensure objectivity and further reduce bias. Disagreements regarding protocol design, data extraction, or synthesis were resolved collaboratively.

Working products, protocol revisions, and inclusion/exclusion decisions are documented and publicly available on GitHub and Zenodo. This study focuses on automotive software architectures, which may limit generalizability to other domains. Directions for future research are outlined in the following section.

9. Conclusion and outlook

9.1. Conclusion

This work addresses the challenge of scheduling and allocation of automotive software tasks on multi-core SoCs. The increasing complexity of modern automotive software, coupled with limited computational resources and growing task dependencies, strives for allocation methods that balance *predictability*, *adaptability*, and *efficiency* at early design stages. Existing approaches typically favor either static allocation, which provides predictable execution behavior but limited flexibility, or dynamic allocation, which improves adaptability at the cost of increased runtime decision-making complexity.

To bridge this gap, we propose a design-space exploration framework that integrates static and dynamic allocation strategies with two scheduling policies: FCFS for arrival-order preservation and low overhead, and PAS for prioritization of event-driven tasks. Static allocation allows controlled analysis of the execution order and per core utilization when determinism is paramount. Dynamic allocation supports runtime adaptation to workload variability and task availability. Across both allocation modes, the choice between FCFS and PAS reflects trade-offs between arrival-order preservation, task waiting time, completion latency,

throughput, and core utilization. The framework incorporates task dependencies and priority attributes into the scheduling and allocation logic. The method selection is guided by metrics such as core utilization, throughput, and task latency, to enable transparent and reproducible evaluation of architectural choices. The presented results demonstrate how RunSoC supports structured, metrics-driven decision-making when selecting scheduling and core allocation strategies for multi-core SoC platforms.

9.2. Outlook

Building on this framework, several research directions emerge:

- **Heterogeneous SoCs**: Extend the framework to support heterogeneous platforms, enabling allocation across cores of differing performance profiles.
- **Adaptive load balancing**: Incorporate runtime load-balancing mechanisms to dynamically redistribute workloads across available cores.
- **Periodicity guard optimization**: Refine the strict periodicity guard to reduce idle latencies, thereby lowering latency without compromising determinism.
- **Fault tolerance**: Integrate redundancy and fail-over strategies so that critical tasks remain schedulable in the event of component or core failures.

CRediT authorship contribution statement

Lucas Mauser: Writing – original draft, Writing – review & editing, Software, Visualization, Resources, Supervision, Project administration, Methodology, Conceptualization, Investigation, Data curation, Formal analysis, Validation; **Xinyu Zhang:** Writing – review & editing, Software, Visualization, Resources, Methodology, Conceptualization, Investigation, Data curation, Formal analysis, Validation; **Daniel Krüger:** Software, Visualization, Resources, Methodology, Validation; **Moritz Wäschle:** Writing – review & editing, Methodology, Conceptualization; **Stefan Wagner:** Writing – review & editing, Supervision, Methodology, Conceptualization.

Declaration of competing interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Detailed data are publicly available on https://github.com/From-SWCtoSemi/RunSoC and https://doi.org/10.5281/zenodo.17334892.

Acknowledgements

We thank the participants of the expert talks for their contributions, engaging input, and thoughtful discussions. Special thanks go to Xinyu Zhang for her dedicated efforts and outstanding achievements during her Master's thesis.

References

Amdahl, G.M., 1967. Validity of the single processor approach to achieving large scale computing capabilities. In: Proceedings of the April 18–20, 1967, Spring Joint Computer Conference, pp. 483–485. https://doi.org/10.1145/1465482.1465560

Ampatzoglou, A., Bibi, S., Avgeriou, P., Verbeek, M., Chatzigeorgiou, A., 2019. Identifying, categorizing and mitigating threats to validity in software engineering secondary studies. Inf. Softw. Technol. 106, 201–230. https://doi.org/10.1016/j.infsof.2018.10.006

Arndt, O.J., Rallapalli, P., Blume, H., 2019. Portable implementations for heterogeneous hardware platforms in autonomous driving systems. In: Big Data Analytics for Cyber-Physical Systems. Elsevier, pp. 113–143. https://doi.org/10.1016/B978-0-12-816637-6.00006-3

AUTOSAR, 2025. Automotive open system architecture. Available at: https://www.autosar.org/. Accessed: 2026-03-14.

Bateni, S., Liu, C., 2019. Predictable data-driven resource management: an implementation using autoware on autonomous platforms. In: 2019 IEEE Real-Time Systems Symposium (RTSS). IEEE, pp. 339–352. https://doi.org/10.1109/RTSS46320.2019.00038

Brereton, P., Kitchenham, B.A., Budgen, D., Turner, M., Khalil, M., 2007. Lessons from applying the systematic literature review process within the software engineering domain. J. Syst. Softw. 80 (4), 571–583. https://doi.org/10.1016/j.jss.2006.07.009

Brown, G., Ravindran, M., Burns, R., Miller, N., 2019. How to deploy AI software to self driving cars. In: Proceedings of the International Workshop on OpenCL, pp. 1. https://doi.org/10.1145/3318170.331818

Cartaxo, B., Pinto, G., Soares, S., 2020. Rapid reviews in software engineering. In: Contemporary Empirical Methods in Software Engineering. Springer, pp. 357–384. https://doi.org/10.1007/978-3-030-32489-6_13

Dendaluce, M., 2015. System-on-chip-based highly integrated powertrain control unit for next-generation electric vehicles: harnessing the potential of hybrid embedded platforms for advanced model-based control algorithms. World Electr. Veh. J. 7 (2), 311–323. https://doi.org/10.3390/wevj7020311

Dong, P., Jiang, Z., Burns, A., Ding, Y., Ma, J., 2020. Build real-time communication for hybrid dual-OS system. J. Syst. Archit. 107, 101774. https://doi.org/10.1016/j.sysarc.2020.101774

Gala, G., Rodriguez, C., Monaco, G., Castillo, J., Fohler, G., Falico, V., Tverdyshev, S., 2022. Monitoring framework to support mixed-criticality applications on multicore platforms. In: 2022 25th Euromicro Conference on Digital System Design (DSD). IEEE, pp. 649–656. https://doi.org/10.1109/DSD57027.2022.00092

Goens, A., Khasanov, R., Castrillon, J., Polstra, S., Pimentel, A., 2016. Why comparing system-level MPSoc mapping approaches is difficult: a case study. In: 2016 IEEE 10th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSOC). IEEE, pp. 281–288. https://doi.org/10.1109/MCSoC.2016.48

Gupta, P., Singh, N.P., Srinivasan, G., 2019. A framework for real-time automotive applications to multicore platform in perspective of autosar. In: 2019 4th International Conference on Recent Trends on Electronics, Information, Communication & Technology (RTEICT). IEEE, pp. 706–709. https://doi.org/10.1109/RTEICT46194.2019.9016689

Hammond, M., Qu, G., Rawashdeh, O.A., 2015. Deploying and scheduling vision based advanced driver assistance systems (ADAS) on heterogeneous multicore embedded platform. In: 2015 Ninth International Conference on Frontier of Computer Science and Technology. IEEE, pp. 172–177. https://doi.org/10.1109/FCST.2015.69

Hemmati, M., Biglari-Abhari, M., Niar, S., 2019. Adaptive vehicle detection for real-time autonomous driving system. In: 2019 Design, Automation & Test in Europe Conference & Exhibition (DATE). IEEE, pp. 1034–1039. https://doi.org/10.23919/DATE.2019.8714818

Hill, M.D., Marty, M.R., 2008. Amdahl's law in the multicore era. Computer 41 (7), 33–38. https://doi.org/10.1109/MC.2008.209

Kelley, , J.E. Jr, Walker, M.R., 1959. Critical-path planning and scheduling. In: Papers Presented at the December 1–3, 1959, Eastern Joint IRE-AIEE-ACM Computer Conference, pp. 160–173. https://doi.org/10.1145/1460299.1460318

Kempf, F., Hoefer, J., Harbaum, T., Becker, J., Fasfous, N., Frickenstein, A., Voegel, H.-J., Friedrich, S., Wittig, R., Matúš, E., et al., 2023. The zuse-ki-mobil ai accelerator soc: overview and a functional safety perspective. In: 2023 Design, Automation & Test in Europe Conference & Exhibition (DATE). IEEE, pp. 1–6. https://doi.org/10.23919/DATE56975.2023.10137257

Khenfri, F., Chaaban, K., Chetto, M., 2020. Efficient mapping of runnables to tasks for embedded AUTOSAR applications. J. Syst. Archit. 110, 101800. https://doi.org/10.1016/j.sysarc.2020.101800

Kitchenham, B., Charters, S., Budgen, D., Brereton, P., Turner, M., Linkman, S., Jorgensen, M., Mendes, E., Visaggio, G., 2007. Guidelines for Performing Systematic Literature Reviews in Software Engineering. Technical Report. Technical report, ver. 2.3 ebse.

Laplante, P.A., 2004. Real-Time Systems Design and Analysis. Wiley Online Library. https://doi.org/10.1002/0471648299

Li, L., Sau, C., Fanni, T., Li, J., Viitanen, T., Christophe, F., Palumbo, F., Raffo, L., Huttunen, H., Takala, J., et al., 2019. An integrated hardware/software design methodology for signal processing systems. J. Syst. Archit. 93, 1–19. https://doi.org/10.1007/s11265-015-1026-0

Mack, J., Gener, S., Hassan, S., Suluhan, H.U., Akoglu, A., 2023. CEDR-API: productive, performant programming of domain-specific embedded systems. In: 2023 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW). IEEE, pp. 16–25. https://doi.org/10.1109/IPDPSW59300.2023.00016

Mauser, L., Wagner, S., 2024. Centralization potential of automotive e/e architectures. J. Syst. Softw. , 112220. https://doi.org/10.1016/j.jss.2024.112220

Mauser, L., Zimmermann, E., Nedvědický, P., Eisenreich, T., Wäschle, M., Wagner, S., 2025. Towards mixed-criticality software architectures for centralized HPC platforms in software-defined vehicles: a systematic literature review. In: European Conference on Software Architecture. Springer, pp. 144–160. https://doi.org/10.1007/978-3-031-36889-9_13

McLean, S.D., Juul Hansen, E.A., Pop, P., Craciunas, S.S., 2022. Configuring ADAS platforms for automotive applications using metaheuristics. Front. Robot. AI 8, 762227. https://doi.org/10.3389/frobt.2021.762227

Moreira, O., Corporaal, H., 2014. Scheduling Real-Time Streaming Applications Onto an Embedded Multiprocessor. Springer. https://doi.org/10.1007/978-3-319-01246-9

Mourão, E., Pimentel, J.F., Murta, L., Kalinowski, M., Mendes, E., Wohlin, C., 2020. On the performance of hybrid search strategies for systematic literature reviews in software engineering. Inf. Softw. Technol. 123, 106294. https://doi.org/10.1016/j.infsof.2020.106294

Navet, N., Simonot-Lion, F., 2017. Automotive Embedded Systems Handbook. CRC press. https://doi.org/10.1201/9780849380273

Ouzzani, M., Hammady, H., Fedorowicz, Z., Elmagarmid, A., 2016. Rayyan–a web and mobile app for systematic reviews. Syst. Rev. 5, 1–10. https://doi.org/10.1186/s13643-016-0384-4

Rettkowski, J., Wehner, P., Schülper, M., Göhringer, D., 2015. A flexible software framework for dynamic task allocation on MPSOCS evaluated in an automotive context. In: International Symposium on Applied Reconfigurable Computing. Springer, pp. 255–266. https://doi.org/10.1007/978-3-319-16214-0_21

Saussard, R., Bouzid, B., Vasiliu, M., Reynaud, R., 2015. Optimal performance prediction of ADAS algorithms on embedded parallel architectures. In: 2015 IEEE 17th International Conference on High Performance Computing and Communications, 2015 IEEE 7th International Symposium on Cyberspace Safety and Security, and 2015IEEE 12th International Conference on Embedded Software and Systems. IEEE, pp. 213–218. https://doi.org/10.1109/HPCC-CSS-ICESS.2015.95

Schnarz, P., Fischer, C., Wietzke, J., Stengel, I., 2015. Secure separation of shared caches in AMP-based mixed criticality systems. SAIEE Afr. Res. J. 106 (2), 93–104. https://doi.org/10.23919/SAIEE.2015.8531890

Skiena, S.S., 2008. The Algorithm Design Manual. Vol. 2. Springer.

Stallings, W., 2016. Computer Organization and Architecture: Designing for Performance. Pearson Education India.

Staron, M., 2021. Automotive Software Architectures. Springer. https://doi.org/10.1007/978-3-030-65939-4

Suluhan, H.U., Gener, S., Fusco, A., Mack, J., Dagli, I., Belviranli, M., Edemen, C., Akoglu, A., 2024. A runtime manager integrated emulation environment for heterogeneous soc design with RISC-v cores. In: 2024 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW). IEEE, pp. 23–30. https://doi.org/10.1109/IPDPSW63119.2024.00013

Tanenbaum, A.S., Bos, H., 2015. Modern Operating Systems. Pearson Education, Inc.

Tatar, G., Bayar, S., 2023. Real-time multi-task ADAS implementation on reconfigurable heterogeneous MPSoc architecture. IEEE Access 11, 80741–80760. https://doi.org/10.1109/ACCESS.2023.3300379

Tatar, G., Bayar, S., Çiçek, İ., 2024. Real-time multi-learning deep neural network on an MPSoc-FPGA for intelligent vehicles: harnessing hardware acceleration with pipeline. IEEE Trans. Intell. Veh. 9 (6), 5021–5032. https://doi.org/10.1109/TIV.2024.3398215

Thulasiraman, K., Swamy, M. N.S., 1992. Graphs: Theory and Algorithms. John Wiley & Sons.

Vogelsang, A., 2020. Feature dependencies in automotive software systems: extent, awareness, and refactoring. J. Syst. Softw. 160, 110458. https://doi.org/10.1016/j.jss.2019.110458

Waez, M. T.B., Wąsowski, A., Dingel, J., Rudie, K., 2014. Synthesis of a reconfiguration service for mixed-criticality multi-core systems: an experience report. In: International Conference on Formal Aspects of Component Software. Springer, pp. 162–180. https://doi.org/10.1007/978-3-319-15317-9_10

Wang, X., Li, H., Xi, W., Zhang, D., Guo, W., 2023. Design and verification of VCU system based on ISO26262. In: Journal of Physics: Conference Series. Vol. 2491. IOP Publishing, p. 012036. https://doi.org/10.1088/1742-6596/2491/1/012036

Warschofsky, R., 2009. Autosar software architecture. Hasso-Plattner-Institute für Softwaresystemtechnik: Potsdam, Germany, 1–12. Available at: https://hpi.de/fileadmin/user_upload/fachgebiete/giese/Ausarbeitungen_AUTOSAR0809/Software_Architecture_Warschofsky.pdf. Accessed: 2026-03-14.